

Project Title

Business Analytics on Nike Sales Data From Raw Data to Insights

Domain

Sales

Objective

- 1.The dataset contained missing values, zero revenues, invalid entries, inconsistent categories, and skewed financial data.
- 2.Preprocessing included cleaning missing values, removing duplicates, correcting data types, and handling outliers.
- 3.Visualizations highlighted trends in revenue, regional performance, profitability, and discount impact.
- 4.The analysis generated recommendations to improve revenue, optimize discounts, and increase profitability

Outcome

The outcome of the project is a cleaned and structured dataset along with improved strategies to enhance profit and revenue trends.

Dataset Information

Source: Kaggle

Year / Timeline: Data collected during 2023 to 2025

Dataset Description:

Order_ID: Contains a unique identifier assigned to each customer transaction. It is used to distinguish individual sales records.

Gender_Category: Represents the customer segment category, including Kids, Women, and Men.

Product_Line: Indicates the product category, such as Soccer, Basketball, Lifestyle, Training, and Running.

Product_Name: Specifies the name of the footwear product selected by the customer.

Size: Represents the shoe size chosen by the customer to ensure proper fit.

Units_Sold: Indicates the number of product units purchased in each transaction.

MRP (Maximum Retail Price): Represents the original listed price of the product before applying any discounts.

Discount_Applied: Indicates the percentage or rate of discount applied to the product.

Revenue: Represents the total income generated from the sale after applying the discount.

Order_Date: Specifies the date on which the customer placed the order.

Sales_Channel: Identifies the mode of purchase, such as Online or Retail.

Region: Represents the geographic location (state or city) in India where the customer made the purchase.

Profit: Represents the financial gain or loss from each transaction after deducting costs from revenue.

Type of Analysis

Descriptive Analysis:

It summarizes the Nike sales dataset by showing overall sales, profit, product performance, and regional distribution to understand the current business performance.

✓ Stages for DA Project

✓ Stage 1 – Problem Definition and Dataset Selection

Define the business problem and expected outcome

Choose dataset and explain its source, size, and features

Import libraries, load dataset

Dataset description (rows, columns, features)

Initial EDA (head, info, describe, shape, null checks, duplicate check)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from google.colab import files
uploaded =files.upload()
```

Choose Files No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving Nike Sales Uncleaned.csv to Nike Sales Uncleaned.csv

Start coding or [generate](#) with AI.

```
df = pd.read_csv('/content/Nike_Sales_Uncleaned.csv')
print (df)
```

	Order_ID	Gender_Category	Product_Line	Product_Name	Size	\
0	2000	Kids	Training	SuperRep Go	M	
1	2001	Women	Soccer	Tiempo Legend	M	
2	2002	Women	Soccer	Premier III	M	
3	2003	Kids	Lifestyle	Blazer Mid	L	
4	2004	Kids	Running	React Infinity	XL	
...	...	...	...	...	...	
2495	4495	Kids	Basketball	Kyrie Flytrap	XL	
2496	4496	Men	Basketball	Kyrie Flytrap	L	
2497	4497	Men	Soccer	Tiempo Legend	7	
2498	4498	Women	Training	ZoomX Invincible	NaN	
2499	4499	Women	Running	Air Zoom	M	

	Units_Sold	MRP	Discount_Applied	Revenue	Order_Date	\
0	NaN	NaN	0.47	0.0	2024-03-09	
1	3.0	4957.93	NaN	0.0	2024-07-09	
2	4.0	NaN	NaN	0.0	NaN	
3	NaN	9673.57	NaN	0.0	04-10-2024	
4	NaN	NaN	NaN	0.0	2024/09/12	
...	...	...	...	...	...	
2495	3.0	NaN	NaN	0.0	2025-05-14	
2496	-1.0	NaN	NaN	0.0	NaN	
2497	NaN	6647.60	0.52	0.0	02-07-2025	
2498	4.0	5358.70	NaN	0.0	11-12-2024	
2499	NaN	5550.99	NaN	0.0	23-11-2024	

	Sales_Channel	Region	Profit
0	Online	bengaluru	-770.45
1	Retail	Hyd	-112.53
2	Retail	Mumbai	3337.34
3	Online	Pune	3376.85
4	Retail	Delhi	187.89
...	...	...	...
2495	Online	Pune	2.97
2496	Online	Hyd	-487.36
2497	Retail	Bangalore	-918.14
2498	Online	Mumbai	3352.29
2499	Retail	Mumbai	2453.57

[2500 rows x 13 columns]

```
print(df.head())
```

	Order_ID	Gender_Category	Product_Line	Product_Name	Size	Units_Sold	\
0	2000	Kids	Training	SuperRep Go	M	NaN	
1	2001	Women	Soccer	Tiempo Legend	M	3.0	
2	2002	Women	Soccer	Premier III	M	4.0	
3	2003	Kids	Lifestyle	Blazer Mid	L	NaN	
4	2004	Kids	Running	React Infinity	XL	NaN	
	MRP	Discount_Applied	Revenue	Order_Date	Sales_Channel	Region	\
0	NaN	0.47	0.0	2024-03-09	Online	bengaluru	
1	4957.93	NaN	0.0	2024-07-09	Retail	Hyd	
2	NaN	NaN	0.0	NaN	Retail	Mumbai	
3	9673.57	NaN	0.0	04-10-2024	Online	Pune	
4	NaN	NaN	0.0	2024/09/12	Retail	Delhi	
Profit							

```
0 -770.45
1 -112.53
2 3337.34
3 3376.85
4 187.89
```

```
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2500 entries, 0 to 2499
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Order_ID              2500 non-null   int64
1   Gender_Category       2500 non-null   object
2   Product_Line          2500 non-null   object
3   Product_Name          2500 non-null   object
4   Size                  1990 non-null   object
5   Units_Sold            1265 non-null   float64
6   MRP                   1246 non-null   float64
7   Discount_Applied      832 non-null    float64
8   Revenue               2500 non-null   float64
9   Order_Date            1884 non-null   object
10  Sales_Channel         2500 non-null   object
11  Region                2500 non-null   object
12  Profit                2500 non-null   float64
dtypes: float64(5), int64(1), object(7)
memory usage: 254.0+ KB
None
```

```
print(df.columns)
```

```
Index(['Order_ID', 'Gender_Category', 'Product_Line', 'Product_Name', 'Size',
       'Units_Sold', 'MRP', 'Discount_Applied', 'Revenue', 'Order_Date',
       'Sales_Channel', 'Region', 'Profit'],
      dtype='object')
```

```
print(df.dtypes)
```

```
Order_ID          int64
Gender_Category   object
Product_Line      object
Product_Name      object
Size              object
Units_Sold        float64
MRP               float64
Discount_Applied  float64
Revenue           float64
Order_Date        object
Sales_Channel     object
Region            object
Profit            float64
dtype: object
```

```
print(df.describe())
```

```
count    Order_ID  Units_Sold      MRP  Discount_Applied  Revenue \
mean    3194.352400    1.482213  6039.863395      0.631022    274.873716
std       749.972638    1.696947  2315.746826      0.365500    2023.773550
min       2000.000000   -1.000000  2006.630000      0.000000   -7561.590000
25%      2534.750000    0.000000  4038.902500      0.320000     0.000000
50%      3192.500000    2.000000  6110.030000      0.615000     0.000000
75%      3841.250000    3.000000  8022.187500      0.962500     0.000000
max       4499.000000    4.000000  9996.220000      1.250000   37169.350000

count    Profit
mean     1376.012848
std      1478.671013
min     -1199.450000
25%       93.532500
50%      1371.385000
75%      2660.645000
max      3999.210000
```

```
print(df.shape)
```

```
(2500, 13)
```

```
print(df.size)
```

32500

```
print(df.isnull().sum())
```

```
Order_ID          0
Gender_Category   0
Product_Line      0
Product_Name      0
Size             510
Units_Sold        1235
MRP              1254
Discount_Applied  1668
Revenue           0
Order_Date        616
Sales_Channel     0
Region           0
Profit           0
dtype: int64
```

```
print(df.duplicated().sum())
```

0

```
print(df.tail)
```

```
<bound method NDFrame.tail of
0      2000      Kids      Training      SuperRep Go      M
1      2001      Women      Soccer      Tiempo Legend      M
2      2002      Women      Soccer      Premier III      M
3      2003      Kids      Lifestyle      Blazer Mid      L
4      2004      Kids      Running      React Infinity      XL
...      ...      ...      ...      ...      ...
2495     4495      Kids      Basketball      Kyrie Flytrap      XL
2496     4496      Men      Basketball      Kyrie Flytrap      L
2497     4497      Men      Soccer      Tiempo Legend      7
2498     4498      Women      Training      ZoomX Invincible      NaN
2499     4499      Women      Running      Air Zoom      M

      Units_Sold      MRP      Discount_Applied      Revenue      Order_Date \
0      NaN      NaN      0.47      0.0      2024-03-09
1      3.0      4957.93      NaN      0.0      2024-07-09
2      4.0      NaN      NaN      0.0      NaN
3      NaN      9673.57      NaN      0.0      04-10-2024
4      NaN      NaN      NaN      0.0      2024/09/12
...      ...      ...      ...      ...      ...
2495     3.0      NaN      NaN      0.0      2025-05-14
2496     -1.0      NaN      NaN      0.0      NaN
2497      NaN      6647.60      0.52      0.0      02-07-2025
2498     4.0      5358.70      NaN      0.0      11-12-2024
2499      NaN      5550.99      NaN      0.0      23-11-2024

      Sales_Channel      Region      Profit
0      Online      bengaluru      -770.45
1      Retail      Hyd      -112.53
2      Retail      Mumbai      3337.34
3      Online      Pune      3376.85
4      Retail      Delhi      187.89
...      ...      ...      ...
2495     Online      Pune      2.97
2496     Online      Hyd      -487.36
2497     Retail      Bangalore      -918.14
2498     Online      Mumbai      3352.29
2499     Retail      Mumbai      2453.57
```

[2500 rows x 13 columns]>

```
print(df.nunique())
```

```
Order_ID          2386
Gender_Category    3
Product_Line       5
Product_Name       20
Size              10
Units_Sold         6
MRP              1244
Discount_Applied   126
Revenue           167
Order_Date        1008
Sales_Channel      2
Region            9
Profit           2494
dtype: int64
```

Stage 2 – Data Cleaning and Pre-processing

Handle missing values (impute or drop)

Handle duplicates

Treat outliers if required

Check skewness and apply transformations

Convert data types if needed

Feature transformations (date parts, derived fields if required for analysis)

```
#Transaction id is created
df["Transaction_ID"] = range(1, len(df)+1)
cols = list(df.columns)

# remove Transaction_ID from current position
cols.remove("Transaction_ID")

# find position of Order_ID
order_index = cols.index("Order_ID") + 1

# insert Transaction_ID next to Order_ID
cols.insert(order_index, "Transaction_ID")

# rearrange dataframe
df = df[cols]
df["Transaction_ID"] = range(1, len(df)+1)
df["Transaction_ID"] = df["Transaction_ID"].astype(str).str.zfill(4)
```

```
df.head()
```

	Order_ID	Transaction_ID	Gender_Category	Product_Line	Product_Name	Size
0	2000	0001	Kids	Training	SuperRep Go	M
1	2001	0002	Women	Soccer	Tiempo Legend	M
2	2002	0003	Women	Soccer	Premier III	M
3	2003	0004	Kids	Lifestyle	Blazer Mid	M
4	2004	0005	Kids	Running	React Airfly	M

```
df["Region"] = df["Region"].replace({
    "bengaluru": "Bangalore",
    "Hyd": "Hyderabad",
    "hyderbad": "Hyderabad"
})
df["Region"] = df["Region"].str.title()
df["Region"].unique()
```

```
array(['Bangalore', 'Hyderabad', 'Mumbai', 'Pune', 'Delhi', 'Kolkata'],
      dtype=object)
```

```
# Convert blank values to NaN
df['Size'] = df['Size'].replace(['', ' ', 'NA', 'null'], np.nan)

# Fill missing values with most frequent value
df['Size'] = df['Size'].fillna(df['Size'].mode()[0])
```

```
# align to center
df.style.set_properties(subset=['size'], **{'text-align': 'center'})
```

```
df["Size"].isnull().sum()
```

```
np.int64(0)
```

```
#Fill missing values with median value in Unit sold column
df.loc[df['Units_Sold'] < 0, 'Units_Sold'] = np.nan
df['Units_Sold'] = df['Units_Sold'].fillna(df['Units_Sold'].median())
```

```
# fill with median values in MRP
df['MRP'] = df['MRP'].fillna(df['MRP'].median())
df['MRP'] = df['MRP'].round(2)
```

```
#Fill missing values with median value in Discount_applied column
df['Discount_Applied'] = df['Discount_Applied'].fillna(df['Discount_Applied'].median())
df['Discount_Applied'] = df['Discount_Applied'].round(2)
```

```
#missing date is filled by median
df['Order_Date'] = pd.to_datetime(df['Order_Date'], errors='coerce')

median_date = df['Order_Date'].median()

df['Order_Date'] = df['Order_Date'].fillna(median_date)
```

```
# Revenue is changed as revenue_corrected
df["Revenue_Corrected"] = df["Units_Sold"] * df["MRP"] * (1 - df["Discount_Applied"])
df["Revenue_Corrected"] = df["Revenue_Corrected"].round(2)

# Estimate Cost (Assuming cost is 70% of MRP)
df["Cost"] = df["Units_Sold"] * df["MRP"] * 0.7
df["Cost"] = df["Cost"].round(2)

# Recalculate Profit
df["Profit_Corrected"] = df["Revenue_Corrected"] - df["Cost"]
df["Profit_Corrected"] = df["Profit_Corrected"].round(2)

# Handle cases where Units_Sold = 0
df.loc[df["Units_Sold"] == 0, "Profit_Corrected"] = 0

print(df.head())
```

	Order_ID	Transaction_ID	Gender_Category	Product_Line	Product_Name	Size	\
0	2000	0001	Kids	Training	SuperRep Go	M	
1	2001	0002	Women	Soccer	Tiempo Legend	M	
2	2002	0003	Women	Soccer	Premier III	M	
3	2003	0004	Kids	Lifestyle	Blazer Mid	L	
4	2004	0005	Kids	Running	React Infinity	XL	

	Units_Sold	MRP	Discount_Applied	Revenue	Order_Date	Sales_Channel	\
0	2.0	6110.03	0.47	0.0	2024-03-09	Online	
1	3.0	4957.93	0.62	0.0	2024-07-09	Retail	
2	4.0	6110.03	0.62	0.0	2024-08-01	Retail	
3	2.0	9673.57	0.62	0.0	2024-08-01	Online	
4	2.0	6110.03	0.62	0.0	2024-08-01	Retail	

	Region	Profit	Revenue_Corrected	Cost	Profit_Corrected
0	Bangalore	-770.45	6476.63	8554.04	-2077.41
1	Hyderabad	-112.53	5652.04	10411.65	-4759.61
2	Mumbai	3337.34	9287.25	17108.08	-7820.83
3	Pune	3376.85	7351.91	13543.00	-6191.09
4	Delhi	187.89	4643.62	8554.04	-3910.42

```
# Creating a new column Profit_Status
df["Profit_Status"] = df["Profit"].apply(
    lambda x: "Profit" if x > 0 else ("Loss" if x < 0 else "Break-even")
)

print(df.head())
```

	Order_ID	Transaction_ID	Gender_Category	Product_Line	Product_Name	Size	\
0	2000	0001	Kids	Training	SuperRep Go	M	
1	2001	0002	Women	Soccer	Tiempo Legend	M	
2	2002	0003	Women	Soccer	Premier III	M	
3	2003	0004	Kids	Lifestyle	Blazer Mid	L	
4	2004	0005	Kids	Running	React Infinity	XL	

	Units_Sold	MRP	Discount_Applied	Revenue	Order_Date	Sales_Channel	\
0	2.0	6110.03	0.47	0.0	2024-03-09	Online	
1	3.0	4957.93	0.62	0.0	2024-07-09	Retail	
2	4.0	6110.03	0.62	0.0	2024-08-01	Retail	
3	2.0	9673.57	0.62	0.0	2024-08-01	Online	
4	2.0	6110.03	0.62	0.0	2024-08-01	Retail	

	Region	Profit	Revenue_Corrected	Cost	Profit_Corrected	\
0	Bangalore	-770.45	6476.63	8554.04	-2077.41	
1	Hyderabad	-112.53	5652.04	10411.65	-4759.61	
2	Mumbai	3337.34	9287.25	17108.08	-7820.83	
3	Pune	3376.85	7351.91	13543.00	-6191.09	
4	Delhi	187.89	4643.62	8554.04	-3910.42	

	Profit_Status
0	Loss
1	Loss
2	Profit

```
3 Profit
4 Profit
```

```
df.to_csv(r"Nike_Sales_Cleaned.csv", index=False)
```

Stage 3 – EDA and Visualizations

Univariate Analysis → distribution of single variables (countplot, histogram, boxplot)

Bivariate Analysis → relation between two variables (scatterplot, barplot, correlation heatmap)

Multivariate Analysis → relation among 3+ variables (pairplot, grouped analysis, pivot tables, advanced plots)

Interpretation MUST with every visualization

Focus on business story not just charts

Each chart must be in separate cells.

✓ Interpretation of your above chart

Explain the Chart.

What is it saying?

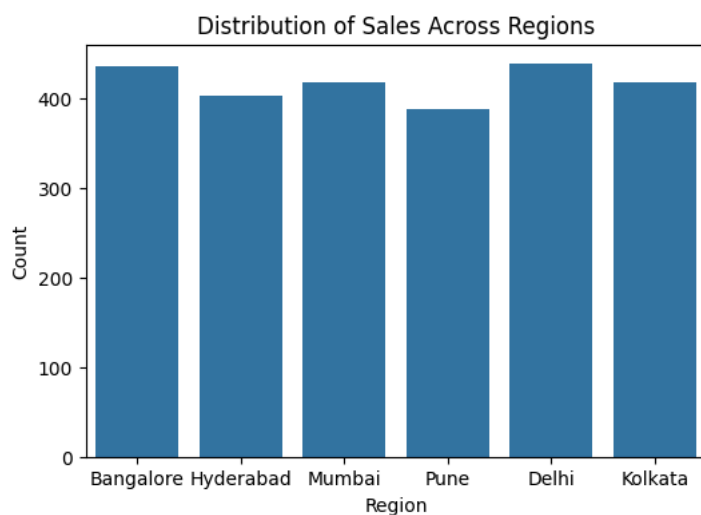
What features you used?

From this what you are showing us?

```
# your code here
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df = pd.read_csv('/content/Nike_Sales_Cleaned.csv')
```

```
# COUNT PLOT (Categorical Variable)
plt.figure(figsize=(6,4))
sns.countplot(x='Region', data=df)
plt.title("Distribution of Sales Across Regions")
plt.xlabel("Region")
plt.ylabel("Count")
plt.show()
```



**** Interpretation of above chart ****

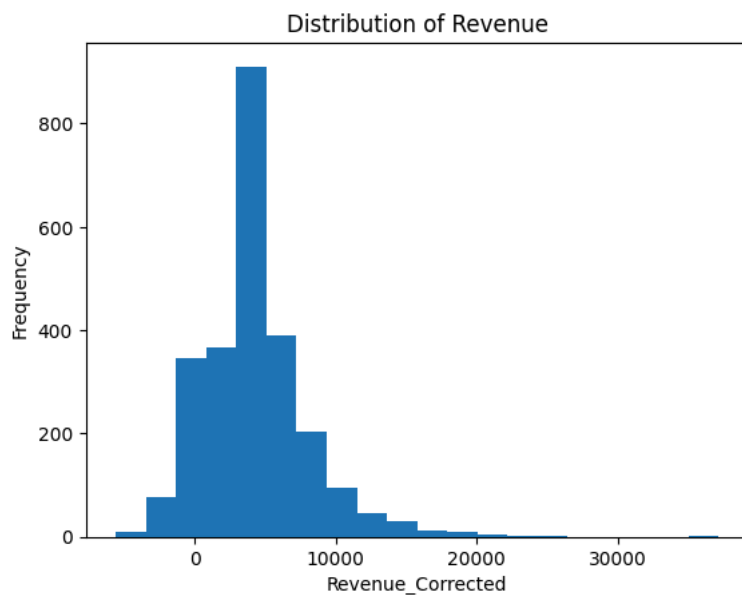
This countplot shows the number of sales transactions across different regions.

It indicates which regions have higher or lower sales activity.

Region feature was used to count and visualize sales distribution.

It helps identify the regions contributing most and least to overall sales.

```
# HISTOGRAM (Numerical Variable)
plt.hist(df['Revenue_Corrected'], bins=20)
plt.title("Distribution of Revenue")
plt.xlabel("Revenue_Corrected")
plt.ylabel("Frequency")
plt.show()
```



**** Interpretation of above chart****

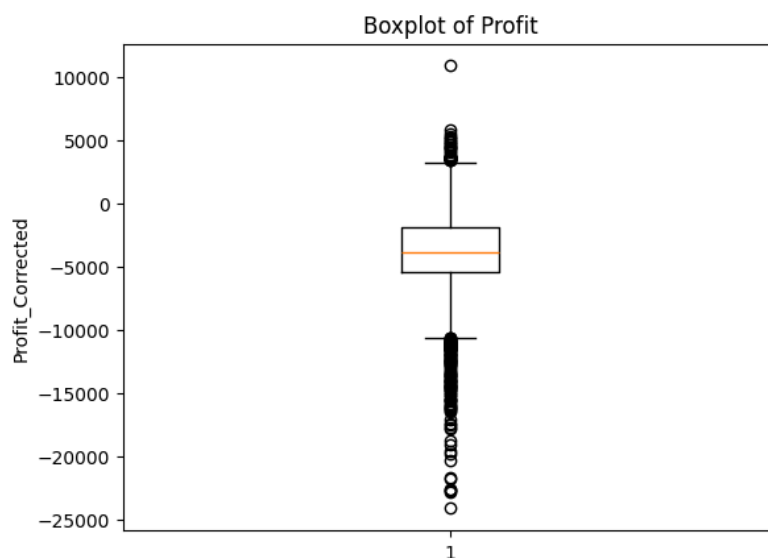
This histogram shows the distribution of revenue values across different ranges.

It indicates how frequently different revenue levels occur in the dataset.

Revenue_Corrected feature was used to create the frequency distribution.

It helps understand revenue patterns, spread, and whether most sales fall in low or high ranges.

```
#BOX PLOT (Outliers + Spread)
plt.boxplot(df['Profit_Corrected'])
plt.title("Boxplot of Profit")
plt.ylabel("Profit_Corrected")
plt.show()
```



**** Interpretation of above chart****

This boxplot shows the distribution, spread, and outliers of profit values.

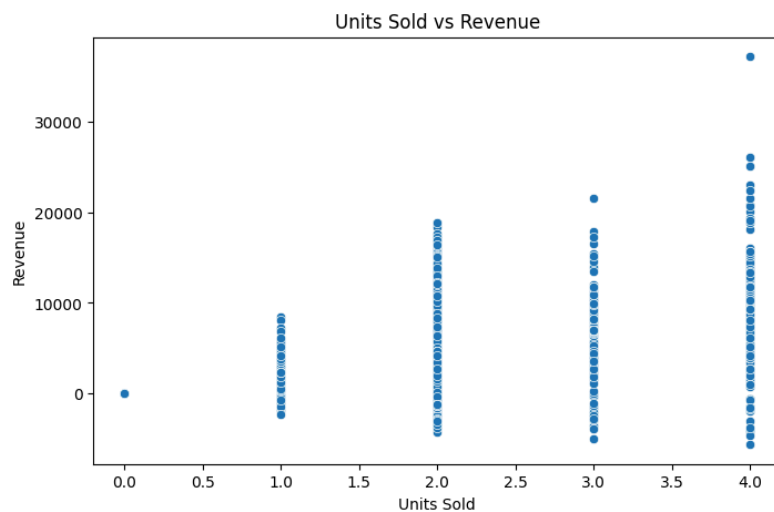
It highlights the median profit, variability, and presence of extreme profit values.

Profit_Corrected feature was used to analyze its distribution.

It helps identify profit consistency and detect unusually high or low profit values.

```
# Scatterplot
plt.figure(figsize=(8,5))
sns.scatterplot(x='Units_Sold', y='Revenue_Corrected', data=df)

plt.title("Units Sold vs Revenue")
plt.xlabel("Units Sold")
plt.ylabel("Revenue")
plt.show()
```



**** Interpretation of above chart****

This scatter plot shows the relationship between units sold and revenue generated.

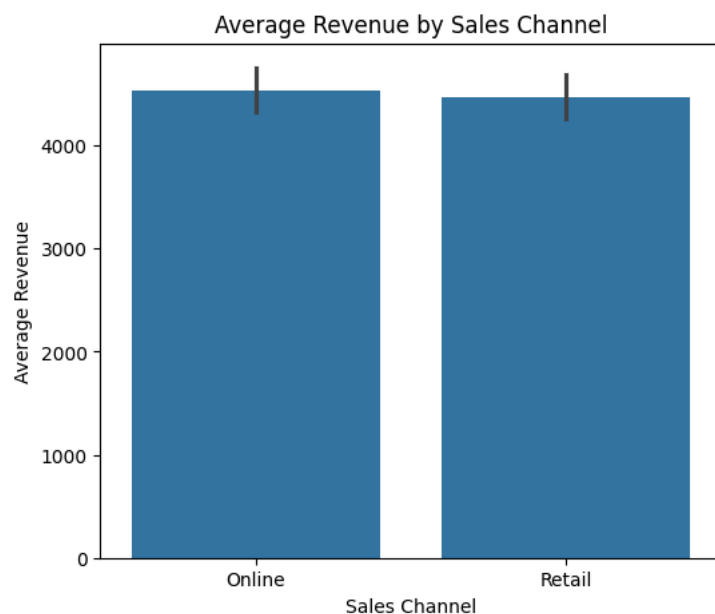
It indicates how revenue changes as the number of units sold increases.

Units_Sold and Revenue_Corrected features were used to analyze their relationship.

It helps identify whether higher sales volume leads to higher revenue and the strength of that relationship.

```
# Barplot
plt.figure(figsize=(6,5))
sns.barplot(x='Sales_Channel', y='Revenue_Corrected', data=df)

plt.title("Average Revenue by Sales Channel")
plt.xlabel("Sales Channel")
plt.ylabel("Average Revenue")
plt.show()
```



Interpretation of above chart

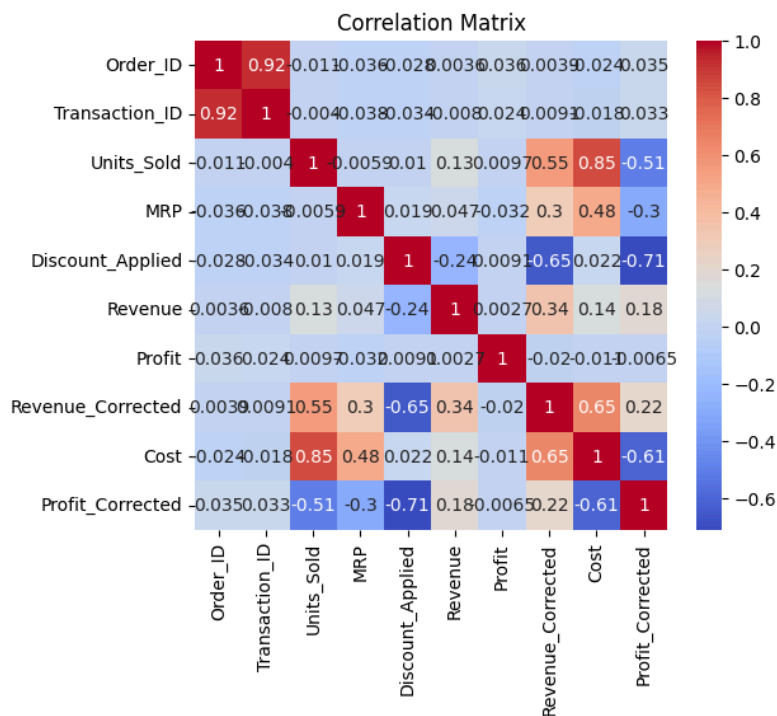
This bar chart shows the average revenue generated by each sales channel.

It compares which sales channel performs better in terms of average revenue.

Sales_Channel and Revenue_Corrected features were used for aggregation and comparison.

It helps identify the most effective sales channel for maximizing revenue.

```
# Correlation Heatmap
plt.figure(figsize=(6,5))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm')
plt.title("Correlation Matrix")
plt.show()
```



Interpretation of above chart

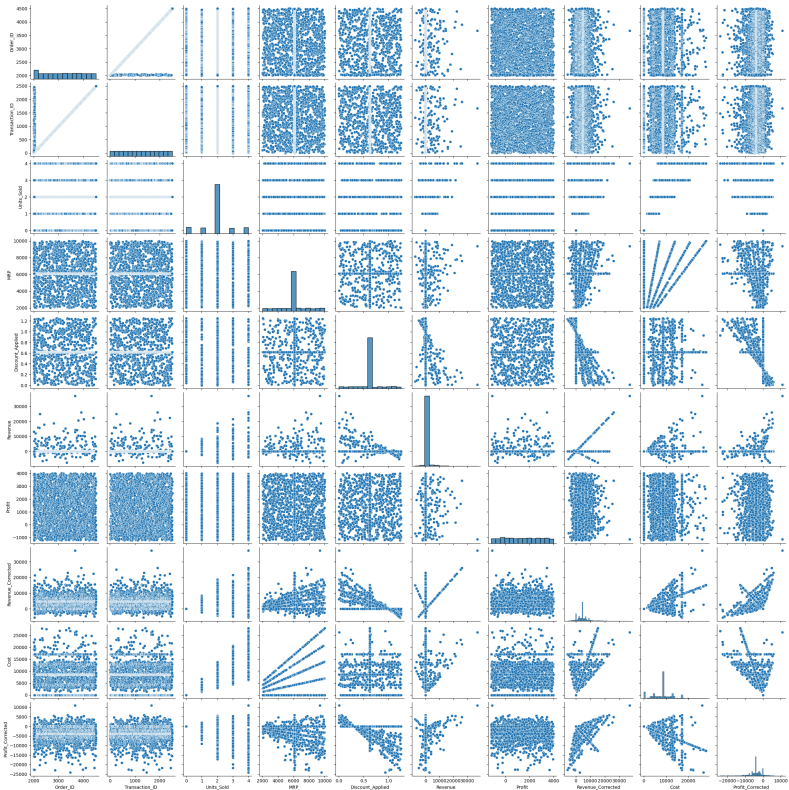
It is heatmap shows the correlation values between all numerical variables in the dataset.

It indicates the strength and direction (positive or negative) of relationships between variables.

All numerical features were used by calculating their correlation matrix.

It helps identify strongly related variables that can impact business decisions or model building.

```
# Pairplot
sns.pairplot(df)
plt.show()
```



Interpretation of above chart

This pairplot displays relationships and distributions among multiple variables in the dataset. It shows how variables are correlated with each other and their individual data distributions. All numerical features in the dataset were automatically used by the pairplot. It helps identify correlations, patterns, and potential clusters across multiple variables

```
# Pivot table
```

```

pivot_table = pd.pivot_table(
    df,
    values='Revenue_Corrected',
    index='Region',
    columns='Sales_Channel',
    aggfunc='sum'
)

print(pivot_table)

```

Sales_Channel	Online	Retail
Region		
Bangalore	1068482.26	810199.67
Delhi	893534.78	1020186.08
Hyderabad	841989.93	947619.71
Kolkata	942514.44	982708.00
Mumbai	991126.36	946044.46
Pune	957180.37	852660.11

Interpretation of above chart

This pivot table summarizes total revenue for each region across different sales channels.

It shows how revenue varies by region and which sales channel contributes more in each region.

Region, Sales_Channel, and Revenue_Corrected features were used with sum aggregation.

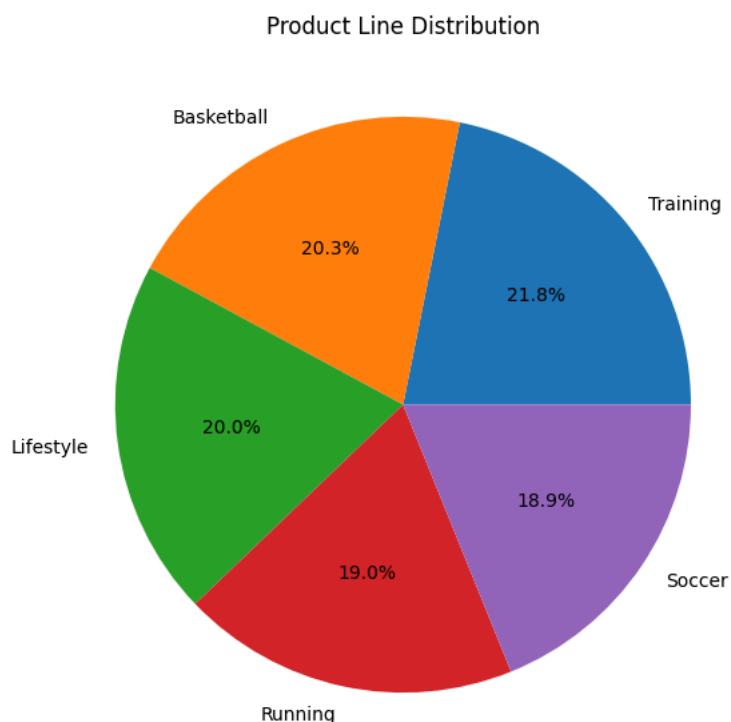
It helps identify the best-performing region and sales channel combinations for revenue generation.

```

# Pie_Chart
product_counts = df['Product_Line'].value_counts()
plt.figure(figsize=(7,7))
plt.pie(product_counts, labels=product_counts.index, autopct='%1.1f%%')

plt.title("Product Line Distribution")
plt.show()

```



```
df.to_csv('Nike_Sales_Cleaned.csv', index=False)
```

Interpretation of above chart

This pie chart clearly represents how each product line contributes to total sales share.

The chart shows the percentage distribution of sales across different product lines.

Product_Line feature was used to count occurrences and visualize their proportions

It highlights which product lines contribute most and least to overall sales distribution

Stage 4 – Documentation, Insights and Presentation

Show a dashboard aligning all the charts in powerbi by connecting python in PowerBI

Summarize findings in plain English (what do the patterns mean)

Highlight 3–5 key insights (trends, anomalies, correlations)

Provide recommendations for business or decision-making

Explain the FINAL STORY WITH THE DASHBOARD

Create a PDF pasting your DASHBOARD, explaining all the above mentioned concepts and submit it

```
# your stage 4 pdf Link
```

```
...
```

```
step1 --> Create a docs that should have all the above mentioned
```

```
step2 --> Save it as "Stage4_doc"
```

```
Step3 --> Download it as PDF
```

```
step4 --> Upload the pdf in your drive
```

```
step5 --> Right click on your pdf in drive
```

```
step6 --> click share and change access into anyone with the link (Viewer)
```

```
step7 --> click copy link and paste it here...
```

```
You are done.
```

```
...
```

```
'\nstep1 --> Create a docs that should have all the above mentioned\nstep2 --> Save it as "Stage4_doc"\nStep3 --> Download it as PDF\nstep4 --> Upload the pdf in your drive\nstep5 --> Right click on your pdf in drive\nstep6 --> click share and change access into anyone with the link (Viewer)\nstep7 --> click copy link and paste it here...\nYou are done.\n'
```

Future Enhancement

Future improvements include real-time data, better interactivity, forecasting, customer analysis, and data security to make the dashboard more effective.

— . .